PRÄSENTATION & PRAXIS-LEITFADEN

Git – Verteilte Versionsverwaltung

Ursprung, Architektur, Vergleich mit SVN und Praxisanwendung mit Python

Diese 20-seitige Präsentationsunterlage vermittelt ein tiefgehendes Verständnis für das Versionsverwaltungssystem **Git**. Beginnend bei der historischen Entstehung im Umfeld des Linux-Kernels und der zugrundeliegenden Snapshot-Philosophie wird Git zunächst mit klassischen zentralisierten Systemen wie Subversion (SVN) verglichen. Anschließend begleiten wir ein durchgängiges Softwareprojekt in Python von der Initialisierung über das Branching und Merging bis hin zur Lösung von Merge-Konflikten und der Synchronisation mit Remote-Servern.

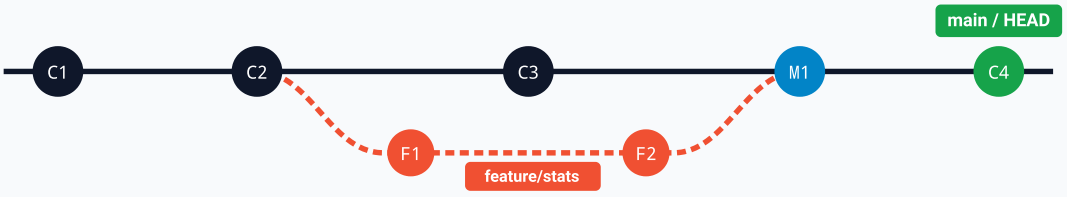


Abbildung 1.1: Schematische Darstellung der nicht-linearen Entwicklungsgeschichte in Git mit Branches und Merge-Commits.

Teil 1: Konzept & Architektur (S. 2–6)

- Warum überhaupt Versionsverwaltung?
- Der historische Ursprung (Linux-Kernel 2005)
- Snapshots statt Delta-Speicherung
- Architekturvergleich: SVN vs. Git
- Die 3-Bäume-Architektur & Dateizustände

Teil 2: Praxis mit Python (S. 7–20)

- Lokales Repository & Python-Basiscode
- Änderungen verfolgen (Status, Diff, Log)
- Branching-Konzepte & Zeiger-Architektur
- Fast-Forward vs. Three-Way-Merge
- Merge-Konflikte lösen & Remote-Workflows

Das Problem: Warum benötigen wir Versionsverwaltung?

Jedes Softwareprojekt durchläuft während seiner Entstehung hunderte oder tausende Änderungen. Neue Funktionen werden hinzugefügt, Fehler behoben und bestehender Code optimiert. Ohne ein formales Versionsverwaltungssystem (*Version Control System*, kurz VCS) versuchen Entwickler oft, ihre Zwischenstände durch manuelles Kopieren von Dateien oder Ordnern zu sichern. Dieses Vorgehen führt bereits bei kleinen Projekten schnell ins Chaos und macht eine strukturierte Zusammenarbeit im Team nahezu unmöglich.

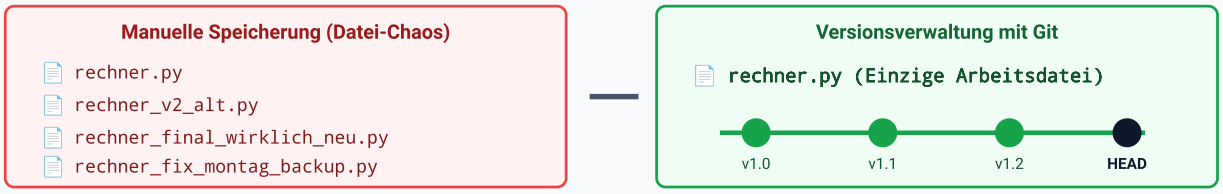


Abbildung 1.2: Statt zahlloser Dateikopien verwaltet Git die gesamte Zeitleiste unsichtbar im Hintergrund.

Ein professionelles Versionsverwaltungssystem protokolliert jede Änderung an einer oder mehreren Dateien über die Zeit hinweg in einer speziellen Datenbank (dem **Repository**). Dadurch beantwortet das System jederzeit vier zentrale Fragen der Softwareentwicklung:

- **Wer** hat eine bestimmte Zeile Code geschrieben oder verändert? (Autorenschaft und Verantwortlichkeit)
- **Wann** genau wurde die Änderung vorgenommen? (Sekundengenaue Zeitstempel für jeden Entwicklungsschritt)
- **Was** genau hat sich zwischen Version A und Version B geändert? (Zeilenweiser Vergleich mittels *Diff*)
- **Warum** wurde diese Änderung durchgeführt? (Dokumentation durch aussagekräftige Commit-Nachrichten)

> **Die Sicherheitsgarantie einer Versionsverwaltung**
>
> Mit einem VCS geht kein bereits gesicherter Arbeitsstand mehr verloren. Wenn eine neue Erweiterung das Programm zum Absturz bringt, kann das Projekt innerhalb von Sekundenbruchteilen auf jeden beliebigen früheren Zustand zurückgesetzt werden. Zudem können mehrere Personen gleichzeitig an derselben Datei arbeiten, ohne sich gegenseitig die Änderungen zu überschreiben.

Der Ursprung von Git: Die Geburt aus der Krise (2005)

Die Entstehungsgeschichte von Git ist eng mit der Entwicklung des **Linux-Kernels** verknüpft – einem der größten Open-Source-Softwareprojekte der Welt. Von 1991 bis 2002 wurden alle Änderungen am Linux-Quellcode ausschließlich in Form von Text-Patches und komprimierten Archivdateien per E-Mail an Linus Torvalds geschickt, der diese manuell prüfte und zusammenführte. Mit dem rasanten Wachstum des Projekts stieß dieses manuelle Verfahren jedoch an seine absoluten Grenzen.

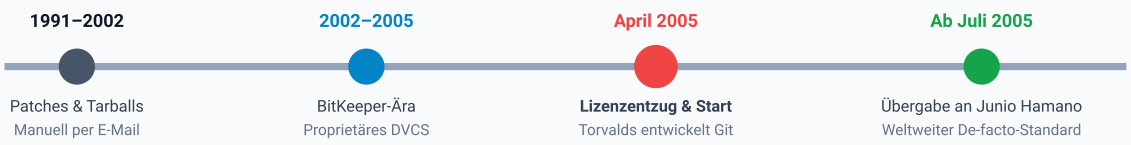


Abbildung 1.3: Meilensteine der historischen Entwicklung von Git durch Linus Torvalds.

Im Jahr 2002 entschied sich Linus Torvalds für den Einsatz des kommerziellen, verteilten Systems **BitKeeper**, das den Kernel-Entwicklern kostenlos zur Verfügung gestellt wurde. Im Frühjahr 2005 kam es jedoch zum Bruch: Nachdem ein Entwickler versucht hatte, das Netzwerkprotokoll von BitKeeper per Reverse-Engineering zu entschlüsseln, zog die Herstellerfirma BitMover die kostenlose Lizenz für die Linux-Community zurück.

Da kein damals verfügbares Open-Source-System (wie CVS oder Subversion) den extremen Anforderungen des Linux-Kernels gerecht wurde, zog sich Linus Torvalds im April 2005 zurück, um ein eigenes System zu schreiben. Innerhalb von nur zehn Tagen stand der erste Prototyp von Git.

Die 4 Designziele von Linus Torvalds

- **Extreme Geschwindigkeit:** Selbst bei zehntausenden Dateien müssen Operationen in Sekundenbruchteilen ablaufen.
- **Verteiltes Design:** Keine Abhängigkeit von einem zentralen Server; volle Offline-Fähigkeit.

Integrität & Nicht-Linearität

- **Starkes Branching:** Tausende parallele Entwicklungszweige müssen mühelos zusammengeführt werden können.
- **Kryptografische Sicherheit:** Schutz der Historie gegen Manipulation durch SHA-1-Prüfsummen.

Snapshots statt Deltas: Wie Git Daten speichert

Der fundamentale Unterschied zwischen Git und fast allen älteren Versionsverwaltungssystemen (wie CVS, Subversion oder Perforce) liegt in der Art und Weise, wie das System über seine Daten denkt. Klassische Systeme arbeiten mit einer **differenzbasierten Speicherung (Delta-Storage)**: Sie speichern die ursprüngliche Basisdatei und merken sich für jede neue Version lediglich die Zeilenänderungen (Deltas) im Vergleich zur Vorversion.

Git verfolgt einen radikal anderen Ansatz: Es betrachtet die Projekthistorie als einen Strom von **Momentaufnahmen (Snapshots)** eines Mini-Dateisystems. Jedes Mal, wenn Sie in Git einen neuen Stand sichern (*Commit*), fotografiert Git gewissermaßen den gesamten Zustand aller Projektdateien zu diesem Zeitpunkt und speichert einen Verweis auf diesen Schnappschuss.

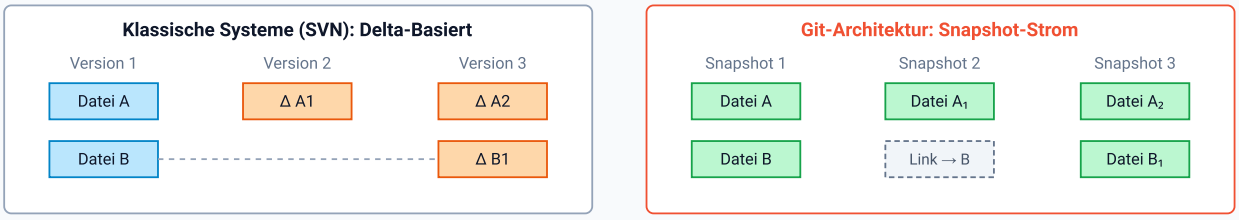


Abbildung 1.4: Während SVN Änderungen (Δ) aufaddiert, speichert Git komplette Dateizustände und verlinkt unveränderte Dateien.

Warum verbraucht Git trotz Snapshots extrem wenig Speicherplatz?

Um effizient zu bleiben, speichert Git unveränderte Dateien bei einem neuen Snapshot **nicht** erneut ab. Stattdessen legt Git lediglich einen winzigen Zeiger (Referenzlink) auf die bereits identisch in der Datenbank vorhandene Datei an. Zusätzlich werden Daten durch zlib komprimiert. Dadurch sind Versionswechsel und das Vergleichen von Zweigen in Git um Größenordnungen schneller als in Systemen, die erst hunderte Delta-Schritte rückwärts oder vorwärts berechnen müssen.

KAPITEL 2 · SYSTEMVERGLEICH

Zentralisiert (SVN) vs. Verteilt (Git)

In der Welt der Versionsverwaltung unterscheiden wir zwei grundlegende Architekturmodelle: **Zentralisierte Systeme (CVCS)** wie Apache Subversion (SVN) oder TFS und **Verteilte Systeme (DVCS)** wie Git oder Mercurial. In einem zentralen System existiert nur ein einziges Repository auf einem Hauptserver; die Entwickler besitzen auf ihren PCs lediglich die aktuell ausgecheckten Arbeitsdateien.



Abbildung 2.1: In Git besitzt jeder Entwickler eine vollwertige Kopie (Klon) des gesamten Repositories inklusive aller Historien.

Kriterium	Subversion (SVN) – Zentralisiert	Git – Verteilt (DVCS)
Repository-Standort	Ausschließlich auf dem zentralen Server.	Jeder Entwickler besitzt ein vollständiges lokales Repository.
Offline-Arbeit	Kaum möglich (kein Commit, kein Log, kein Branching ohne Netzwerk).	Vollständig möglich; alle Operationen finden primär lokal statt.
Geschwindigkeit	Langsamer, da fast jeder Befehl über das Netzwerk kommuniziert.	Extrem schnell, da auf die lokale Festplatte zugegriffen wird.
Branching & Merging	Schwergewichtig (kopiert komplette Verzeichnisse auf dem Server).	Leichtgewichtig (erstellt lediglich einen 41 Byte großen Zeiger).
Ausfallsicherheit	Single Point of Failure: Fällt der Server aus, steht das Team still.	Jeder Klon auf einem Entwickler-PC ist ein vollwertiges Backup.

Die Drei-Bäume-Architektur und die vier Dateizustände

Um Git souverän zu bedienen, muss man verstehen, dass ein lokales Git-Projekt intern aus **drei logischen Bereichen (Bäumen)** besteht, zwischen denen Dateien hin- und herbewegt werden. Anders als bei SVN, wo Änderungen direkt vom Arbeitsordner auf den Server geschrieben werden, schaltet Git mit der sogenannten **Staging Area** (auch *Index* genannt) eine Vorbereitungszone dazwischen.

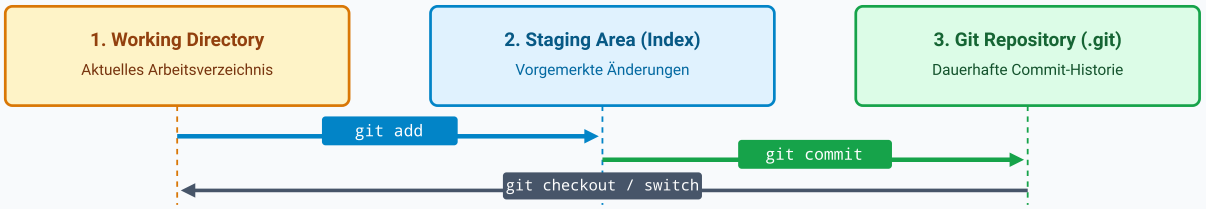


Abbildung 3.1: Datenfluss zwischen Arbeitsverzeichnis, Staging Area und der lokalen Git-Datenbank.

Aus dieser Dreiteilung ergeben sich genau **vier Zustände**, in denen sich jede Datei innerhalb Ihres Projektordners befinden kann:

1. Untracked & 2. Unmodified

- **Untracked (Unbeobachtet):** Die Datei wurde neu erstellt, ist Git aber noch völlig unbekannt.
- **Unmodified (Unverändert):** Die Datei ist bereits in Git gespeichert und wurde seit dem letzten Commit nicht mehr verändert.

3. Modified & 4. Staged

- **Modified (Geändert):** Sie haben die Datei im Editor bearbeitet, sie aber noch nicht für den nächsten Commit vorgemerkt.
- **Staged (Vorgemerkt):** Die geänderte Datei wurde mit `git add` in die Staging Area gelegt und wartet auf den nächsten Snapshot.

Konfiguration und Initialisierung eines Repositories

Ab dieser Seite begleiten wir ein konkretes Softwareprojekt in **Python**. Wir entwickeln ein kleines Statistik- und Taschenrechner-Modul namens `rechner.py`. Bevor wir auf einem neuen Computer das erste Mal mit Git arbeiten, hinterlegen wir einmalig unseren Namen und unsere E-Mail-Adresse. Diese Daten bettet Git unauslöschlich in jeden von uns erzeugten Commit ein.

```
      # 1. Einmalige globale Identitätskonfiguration festlegen $ git config --global
user.name "Max Mustermann" $ git config --global user.email "max.mustermann@example.de"
# Standardnamen für den Hauptzweig auf 'main' setzen $ git config --global
init.defaultBranch main # 2. Neuen Projektordner anlegen und hineinwechseln $ mkdir
python-rechner $ cd python-rechner # 3. Leeres Git-Repository im aktuellen Ordner
initialisieren $ git init Initialized empty Git repository in /home/max/python-
rechner/.git/
```

Durch den Befehl `git init` erzeugt Git ein verstecktes Unterverzeichnis namens `.git`. In diesem Ordner befinden sich ab sofort alle Objektdatenbanken, Konfigurationsdateien, Branch-Zeiger und Commit-Protokolle unseres Projekts. Im übergeordneten Ordner arbeiten wir ganz normal mit unseren Python-Quelldateien.

Die Bedeutung der Datei .gitignore in Python-Projekten

Bei der Ausführung von Python-Skripten erzeugt der Interpreter automatisch Bytecode-Cache-Dateien (z. B. im Ordner `__pycache__/` oder Dateien mit der Endung `.pyc`) sowie virtuelle Umgebungen (`venv/`). Solche generierten Dateien gehören **niemals** in die Versionsverwaltung! Legen Sie daher im Hauptverzeichnis eine Textdatei namens `.gitignore` an und tragen Sie dort die Muster ein, die Git dauerhaft ignorieren soll:

```
      # Inhalt der Datei .gitignore __pycache__/ *.py[cod] .venv/ .env
```

Das erste Python-Skript erstellen und sichern (Add & Commit)

Nun erstellen wir im Projektordner `python-rechner` unsere erste Programmdatei `rechner.py`. Zu Beginn soll unser Modul lediglich zwei einfache Grundrechenarten – die Addition und die Subtraktion zweier Zahlen – bereitstellen sowie einen kurzen Testaufruf im Hauptprogramm enthalten.

```
# Datei: rechner.py (Version 1 - Basisimplementierung) def addiere(a, b): """Gibt die Summe von a und b zurück.""" return a + b def subtrahiere(a, b): """Gibt die Differenz von a und b zurück.""" return a - b if __name__ == "__main__": print("=== Python Rechner v1.0 ===") print(f"10 + 5 = {addiere(10, 5)}") print(f"10 - 5 = {subtrahiere(10, 5)}")
```

Nachdem wir die Datei im Editor gespeichert haben, prüfen wir mit `git status` den aktuellen Zustand unseres Arbeitsverzeichnisses. Anschließend befördern wir die Datei mit `git add` in die Staging Area und erzeugen mit `git commit` unseren ersten dauerhaften Schnappschuss:

```
# 1. Zustand prüfen: Git erkennt die neue Datei als 'Untracked' $ git status On branch main No commits yet Untracked files: (use "git add <file>..." to include in what will be committed) rechner.py # 2. Datei in die Staging Area aufnehmen $ git add rechner.py # 3. Ersten Snapshot mit aussagekräftiger Beschreibung (Flag -m) speichern $ git commit -m "Initialer Commit: Basis-Rechner mit Addition und Subtraktion" [main (root-commit) a1b2c3d] Initialer Commit: Basis-Rechner mit Addition und Subtraktion 1 file changed, 13 insertions(+) create mode 100644 rechner.py
```

Die Rückmeldung von Git zeigt uns wichtige Details: Auf dem Zweig `main` wurde der Wurzel-Commit mit dem Kurz-Hash `a1b2c3d` erzeugt, der genau eine Datei und 13 eingefügte Codezeilen umfasst.

Den Code erweitern und Änderungen mit git diff analysieren

Im nächsten Entwicklungsschritt erweitern wir unser Programm `rechner.py` um eine neue Funktion `multipliziere(a, b)` und passen die Ausgabe im Hauptblock entsprechend an. Sobald wir die Datei speichern, wechselt ihr Status in Git automatisch von *Unmodified* auf *Modified*.

```
# Datei: rechner.py (Erweiterung um Multiplikation) def addiere(a, b): return a + b def subtrahiere(a, b): return a - b def multipliziere(a, b): """Gibt das Produkt von a und b zurück.""" return a * b if __name__ == "__main__": print("=== Python Rechner v1.1 ===") print(f"10 + 5 = {addiere(10, 5)}") print(f"10 - 5 = {subtrahiere(10, 5)}") print(f"10 * 5 = {multipliziere(10, 5)}")
```

Bevor wir die Änderung sichern, möchten wir genau kontrollieren, welche Zeilen wir seit dem letzten Commit verändert haben. Dafür bietet Git das mächtige Werkzeug `git diff`:

```
$ git diff diff --git a/rechner.py b/rechner.py index 84c91a2..f39d12b 100644 --- a/rechner.py +++ b/rechner.py @@ -7,5 +7,10 @@ def subtrahiere(a, b): return a - b +def multipliziere(a, b): + """Gibt das Produkt von a und b zurück.""" + return a * b + if __name__ == "__main__": - print("=== Python Rechner v1.0 ===") + print("=== Python Rechner v1.1 ===") print(f"10 + 5 = {addiere(10, 5)}") print(f"10 - 5 = {subtrahiere(10, 5)}") + print(f"10 * 5 = {multipliziere(10, 5)}") # Änderung vormerken und als zweiten Commit speichern $ git add rechner.py $ git commit -m "Feature: Multiplikationsfunktion hinzugefügt"
```

In der Diff-Ansicht kennzeichnet ein Minuszeichen (-) gelöschte oder ersetzte Zeilen (wie die Versionsnummer v1.0), während ein Pluszeichen (+) alle neu hinzugekommenen Zeilen markiert.

Die Projekthistorie erkunden: git log und SHA-1-Hashes

Jeder Commit in Git wird durch eine 40-stellige hexadezimale Prüfsumme (einen **SHA-1-Hash** wie `e4f9a2b...`) eindeutig identifiziert. Dieser Hash wird aus dem gesamten Dateiinhalt, dem Autor, dem Datum, der Commit-Nachricht sowie – ganz entscheidend – dem Hash des direkten Vorgänger-Commits berechnet. Dadurch entsteht eine kryptografisch verkettete Historie (ähnlich einer Blockchain), bei der nachträgliche Manipulationen alter Versionen sofort auffallen würden.



Abbildung 4.1: In Git zeigt jeder neue Commit auf seinen Vorgänger (Parent-Pointer).

Mit dem Befehl `git log` blättern wir durch das chronologische Tagebuch unseres Projekts. Durch zusätzliche Parameter lässt sich die Darstellung für große Projekte extrem kompakt formatieren:

```
      # 1. Detaillierte Standardansicht der Commit-Historie $ git log commit
f8e9d0c1b2a34567890fedcba9876543210abcde (HEAD -> main) Author: Max Mustermann
<max.mustermann@example.de> Date: Thu Oct 22 09:15:42 2026 +0200 Feature:
Multiplikationsfunktion hinzugefügt commit a1b2c3d4e5f678901234567890abcdef12345678
Author: Max Mustermann <max.mustermann@example.de> Date: Thu Oct 22 09:02:10 2026 +0200
Initialer Commit: Basis-Rechner mit Addition und Subtraktion # 2. Kompakte Einzeiler-
Ansicht mit ASCII-Verzweigungsgraph $ git log --oneline --graph --decorate --all *
f8e9d0c (HEAD -> main) Feature: Multiplikationsfunktion hinzugefügt * a1b2c3d Initialer
Commit: Basis-Rechner mit Addition und Subtraktion
```

Das Konzept von Branches: Warum Zweige in Git so mächtig sind

Das Erstellen von Entwicklungszweigen (**Branching**) ermöglicht es, von der stabilen Hauptentwicklungslinie abzubiegen und an einer neuen Funktion oder einer Fehlerbehebung zu arbeiten, ohne den funktionierenden Hauptcode zu gefährden. In älteren Systemen wie Subversion erfordert das Anlegen eines Branches das physische Kopieren aller Projektdateien in ein neues Serververzeichnis – was je nach Projektgröße viele Sekunden oder Minuten dauert.

In Git hingegen ist ein Branch **kein Ordner mit Dateikopien**, sondern lediglich ein winziger, beweglicher **Zeiger (Pointer)** auf einen bestimmten Commit. Wenn Sie einen neuen Branch anlegen, schreibt Git nur eine einzige 41 Byte große Datei mit der SHA-1-Adresse des aktuellen Commits. Das dauert weniger als eine Millisekunde!



Abbildung 5.1: Zwei Branches zeigen auf denselben Commit; der Spezialzeiger HEAD markiert den aktuell aktiven Zweig.

Woher weiß Git, auf welchem Branch wir uns gerade befinden?

Dafür besitzt Git einen speziellen Zeiger namens **HEAD**. Man kann sich HEAD wie einen Lesezeichen-Pfeil vorstellen, der immer auf denjenigen lokalen Branch zeigt, den Sie gerade ausgecheckt haben. Wenn Sie den Branch wechseln, verschiebt Git lediglich den HEAD-Zeiger auf den anderen Branch-Namen und aktualisiert Ihr Arbeitsverzeichnis entsprechend.

Einen Feature-Branch erstellen und Python-Code isoliert erweitern

Wir möchten unseren Python-Rechner nun um eine Divisionsfunktion erweitern, die auch eine Fehlerbehandlung für die Division durch Null enthält. Um unseren stabilen Hauptzweig `main` während der Entwicklung sauber zu halten, legen wir einen neuen Branch namens `feature-division` an und wechseln direkt dorthin:

```
# Neuen Branch 'feature-division' erstellen und sofort aktivieren # (Alternativ in älteren Git-Versionen: git checkout -b feature-division) $ git switch -c feature-division Switched to a new branch 'feature-division'
```

Nun öffnen wir `rechner.py`, ergänzen die Funktion `dividiere(a, b)` und sichern diesen Stand auf unserem neuen Feature-Branch:

```
# Ergänzung in rechner.py auf dem Branch 'feature-division' def dividiere(a, b): """Dividiert a durch b mit Schutz vor Division durch Null.""" if b == 0: raise ValueError("Fehler: Division durch Null ist nicht erlaubt!") return a / b
```

```
$ git commit -a -m "Feature: Sichere Divisionsfunktion implementiert" [feature-division 7c4a91e] Feature: Sichere Divisionsfunktion implementiert 1 file changed, 6 insertions(+)
```



Abbildung 5.2: Der Zeiger 'feature-division' ist um einen Commit weitergerückt, während 'main' unverändert auf f8e9d verharrt.

Fast-Forward-Merge: Das Zusammenführen ohne Abzweigung

Nachdem wir die Divisionsfunktion auf dem Zweig `feature-division` ausgiebig getestet haben, möchten wir sie in unseren Hauptzweig `main` übernehmen. Da auf dem Branch `main` seit unserer Abzweigung **keine neuen Commits** hinzugekommen sind, liegt der Commit `7c4a91e` in direkter Linie vor `main`.

In dieser Situation führt Git einen sogenannten **Fast-Forward-Merge (Schnellvorlauf)** durch: Es muss kein neuer Merge-Commit erzeugt werden. Stattdessen schiebt Git den Zeiger `main` einfach entlang der Kette nach vorne auf denselben Commit, auf den auch `feature-division` zeigt.

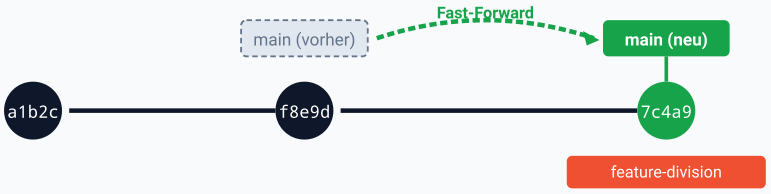


Abbildung 5.3: Beim Fast-Forward-Merge rückt der Zeiger 'main' einfach auf den neuesten Stand des Feature-Branches vor.

Um einen Merge durchzuführen, wechselt man **immer zuerst in den Ziel-Branch** (hier `main`), der die Änderungen aufnehmen soll, und führt anschließend `git merge` aus:

```
      # 1. Zurück auf den Hauptzweig 'main' wechseln $ git switch main Switched to
branch 'main' # 2. Änderungen aus 'feature-division' in 'main' integrieren $ git merge
feature-division Updating f8e9d0c..7c4a91e Fast-forward rechner.py | 6 ++++++ 1 file
changed, 6 insertions(+) # 3. Den nicht mehr benötigten Feature-Branch sauber löschen $
git branch -d feature-division Deleted branch feature-division (was 7c4a91e).
```

Three-Way-Merge: Echte parallele Entwicklung zusammenführen

Im Alltag eines Entwicklerteams bleibt der Hauptzweig `main` selten stehen, während an einem Feature gearbeitet wird. Nehmen wir an, wir zweigen einen neuen Branch `feature-potenz` ab, um eine Funktion `potenziere(a, b)` zu schreiben. Gleichzeitig fügt ein Kollege auf dem `main`-Branch eine Dokumentationszeile am Dateianfang hinzu. Nun sind die beiden Entwicklungsstränge echt auseinandergelaufen (divergiert).

Ein einfacher Schnellvorlauf ist jetzt unmöglich. Git führt daher einen **Three-Way-Merge (Drei-Wege-Merge)** durch: Es betrachtet drei Snapshots – den gemeinsamen Vorfahren (**C3**) sowie die beiden aktuellen Spitzen von `main` (**C4**) und `feature-potenz` (**F1**) – und kombiniert deren Änderungen in einem brandneuen **Merge-Commit (M1)**, der zwei Eltern-Commits besitzt.

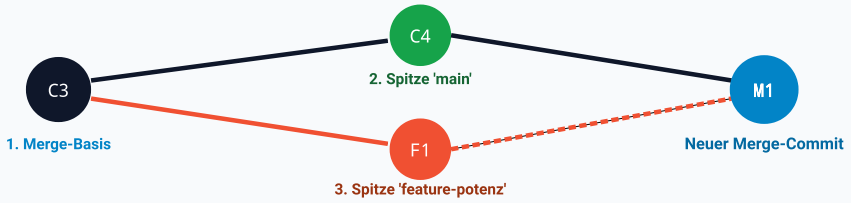


Abbildung 5.4: Git kombiniert die beiden Zweige C4 und F1 mithilfe ihrer gemeinsamen Basis C3 zu einem neuen Merge-Commit M1.

```
      # Auf dem Branch 'main' das zusammengeführte Ergebnis erzeugen $ git switch main
$ git merge feature-potenz Merge made by the 'ort' strategy. rechner.py | 5 +++++ 1
file changed, 5 insertions(+)
```

Merge-Konflikte verstehen und im Python-Code auflösen

Solange zwei Branches unterschiedliche Stellen einer Datei verändern, führt Git diese beim Merge vollautomatisch zusammen. Was passiert jedoch, wenn zwei Entwickler auf verschiedenen Branches **exakt dieselbe Zeile Code** unterschiedlich abändern? In diesem Fall kann Git nicht erraten, welche Version fachlich richtig ist, hält den Merge-Vorgang an und meldet einen **Merge-Konflikt**.

Szenario: Auf dem Branch `main` wurde die Begrüßungszeile in `rechner.py` auf `"=== Profi-Rechner v2.0 ==="` geändert. Parallel dazu wurde genau dieselbe Zeile auf dem Branch `design-update` in `"*** Mathe-Modul Edition ***"` umbenannt. Beim Aufruf von `git merge design-update` markiert Git die Konfliktstelle direkt in der Python-Datei:

```
# Datei rechner.py mit eingefügten Git-Konfliktmarkern if __name__ == "__main__": <<<<<<< HEAD print("=== Profi-Rechner v2.0 ===") ======= print("*** Mathe-Modul Edition ***") >>>>>>> design-update print(f"10 + 5 = {addiere(10, 5)}")
```

Die Konfliktmarker sind leicht zu lesen: Zwischen `<<<<<<< HEAD` und dem Gleichheitszeichen-Trenner `=======` steht der Code unseres aktuellen Ziel-Branches (`main`). Zwischen `=======` und `>>>>>>> design-update` steht der konkurrierende Code des einzufügenden Zweigs. Um den Konflikt zu lösen, entscheiden wir uns für den gewünschten Text, **löschen alle drei Marker-Zeilen vollständig** und schließen den Merge mit einem Commit ab:

```
# 1. Bereinigter Python-Code in rechner.py (ohne Marker!) if __name__ == "__main__": print("=== Profi-Rechner v2.0 (Mathe-Modul Edition) ===") print(f"10 + 5 = {addiere(10, 5)}") # 2. Datei durch 'git add' als aufgelöst markieren und Merge abschließen $ git add rechner.py $ git commit -m "Merge branch 'design-update': Titelzeile kombiniert"
```

Code hochladen und teilen: Remote-Repositories (GitHub & GitLab)

Bisher haben alle Operationen ausschließlich lokal auf unserem eigenen Rechner stattgefunden. Um unser Python-Projekt mit Kollegen zu teilen oder in der Cloud zu sichern, verknüpfen wir unser lokales Repository mit einem **Remote-Repository** auf Plattformen wie **GitHub**, **GitLab** oder **Bitbucket**. Der Standardname, den Git für diesen entfernten Server vergibt, lautet per Konvention `origin`.

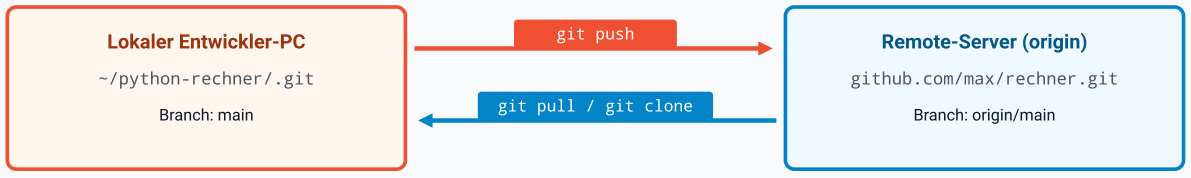


Abbildung 7.1: Austausch von Commits zwischen lokalem Repository und Remote-Server über das Netzwerk.

So verknüpfen wir unser bestehendes Python-Projekt mit einem neu angelegten Repository auf einem Git-Server und laden unseren Code mit `git push` hoch:

```
# 1. Adresse des Remote-Servers unter dem Kurznamen 'origin' registrieren $ git remote add origin https://github.com/max-mustermann/python-rechner.git # 2. Lokalen Branch 'main' hochladen und mit '-u' dauerhaft verknüpfen $ git push -u origin main Enumerating objects: 12, done. Counting objects: 100% (12/12), done. Writing objects: 100% (12/12), 1.42 KiB | 1.42 MiB/s, done. To https://github.com/max-mustermann/python-rechner.git * [new branch] main -> main branch 'main' set up to track 'origin/main'.
```

Änderungen abrufen (Fetch vs. Pull) und der Pull-Request-Workflow

Wenn mehrere Entwickler am selben Python-Projekt arbeiten, müssen regelmäßig die neuesten Änderungen der Kollegen vom Server heruntergeladen werden. Git unterscheidet hierbei strikt zwischen zwei Befehlen, die oft verwechselt werden: `git fetch` und `git pull`.

`git fetch` (Sicherer Download)

Lädt alle neuen Commits und Branches vom Server in Ihre lokale Datenbank herunter, **verändert aber Ihre Arbeitsdateien nicht**. Sie können die Änderungen der Kollegen erst in Ruhe mit `git diff main origin/main` prüfen, bevor Sie diese manuell zusammenführen.

`git pull` (Download + Merge)

Ist eine direkte Abkürzung für die Kombination aus `git fetch` gefolgt von einem sofortigen `git merge`. Die neuesten Commits vom Server werden heruntergeladen und direkt in Ihren aktuell geöffneten Arbeits-Branch integriert.

In professionellen Softwareteams arbeitet niemand direkt auf dem geschützten Hauptzweig `main`. Stattdessen kommt der **Feature-Branch-Workflow mit Pull Requests** (bei GitLab *Merge Requests* genannt) zum Einsatz:



Abbildung 7.2: Qualitätssicherung im Team durch den fünfstufigen Pull-Request-Workflow.

Änderungen zwischenspeichern (Stash) und Fehler rückgängig machen

In der täglichen Praxis treten häufig zwei Situationen auf: Entweder man muss mitten in einer unfertigen Programmierarbeit dringend den Branch wechseln (wofür das Arbeitsverzeichnis sauber sein muss), oder man hat versehentlich einen fehlerhaften Commit erzeugt, der korrigiert werden muss. Für beides bietet Git elegante Sicherheitsnetze.



Abbildung 8.1: Zwischenspeichern mit 'git stash' (links) und sicheres Rückgängigmachen mit 'git revert' (rechts).

```
# 1. Unfertige Änderungen im Arbeitsverzeichnis kurzzeitig weglegen $ git stash
Saved working directory and index state WIP on main: 7c4a91e # ... anderen Branch
bearbeiten, danach zurückkehren und Änderungen herausholen: $ git stash pop # 2. Noch
nicht vorgemerkte Änderungen an einer Datei komplett verwerfen $ git restore rechner.py
# 3. Einen fehlerhaften Commit (z.B. 7c4a91e) durch Gegen-Commit neutralisieren #
(Sicher für bereits gepushte Commits, da die Historie nicht gelöscht wird!) $ git
revert 7c4a91e
```

KAPITEL 9 · KOMPAKTE REFERENZ

Das große Git-Spickzettel-Kompendium (Cheat Sheet)

Die folgende Übersicht fasst die wichtigsten Befehle für den täglichen Einsatz in der Softwareentwicklung thematisch geordnet zusammen:

Git-Befehl	Kategorie	Wirkung & Beschreibung
`git init`	Setup	Initialisiert ein neues, leeres Git-Repository im aktuellen Ordner.
`git clone <url>`	Setup	Lädt ein bestehendes Remote-Repository komplett samt Historie herunter.
`git status`	Basis-Alltag	Zeigt geänderte, vorgemerkte und unbeobachtete Dateien an.
`git add <datei>` / `git add .`	Basis-Alltag	Befördert Änderungen in die Staging Area für den nächsten Commit.
`git commit -m "Text"`	Basis-Alltag	Speichert den Inhalt der Staging Area dauerhaft als Snapshot.
`git diff`	Analyse	Zeigt zeilenweise Unterschiede zwischen Arbeitsordner und Staging Area.
`git log --oneline --graph`	Analyse	Stellt die Commit-Historie kompakt mit Verzweigungsbaum dar.
`git switch -c <name>`	Branching	Erstellt einen neuen Branch und wechselt sofort auf diesen Zweig.
`git merge <branch>`	Branching	Führt den angegebenen Branch in den aktuell aktiven Branch zusammen.
`git push -u origin <branch>`	Remote	Lädt lokale Commits auf den verknüpften Server hoch.
`git pull`	Remote	Holt Änderungen vom Server und führt sie direkt lokal zusammen.
`git restore <datei>`	Rettung	Verwirft lokale Änderungen im Arbeitsverzeichnis auf den letzten Commit.

> **Tipp für die Kommandozeile: Hilfe direkt abrufen**
>
> Wenn Sie sich bei den Optionen eines Befehls unsicher sind, öffnet der Aufruf `git <befehl> --help` (also z. B. `git merge --help`) jederzeit das vollständige, lokal installierte Handbuch inklusive Beispielen.

KAPITEL 9 · ZUSAMMENFASSUNG & QUALITÄTSSTANDARDS

Goldene Regeln (Best Practices) für den erfolgreichen Git-Einsatz

Ein Versionsverwaltungssystem ist nur so gut wie die Disziplin, mit der es im Projektalltag genutzt wird. Wer die folgenden vier Grundregeln beherzigt, hält die Projekthistorie auch über Jahre hinweg sauber, nachvollziehbar und fehlerfrei:

1. Atomare Commits schneiden

Packen Sie niemals fünf verschiedene Themen in einen einzigen Riesen-Commit. Jeder Commit sollte genau **eine logisch abgeschlossene Änderung** umfassen, sodass er sich bei Fehlern isoliert zurücknehmen lässt.

2. Sprechende Commit-Nachrichten

Vermeiden Sie nichtssagende Kommentare wie *"Bugfix"* oder *"Update"*. Beschreiben Sie in einer kurzen Betreffzeile (max. 50 Zeichen), **was** und **warum** geändert wurde.

3. Keine generierten Dateien oder Secrets

Kompilate, Logdateien, große Datenbanken und vor allem **Passwörter oder API-Schlüssel** gehören niemals in ein Git-Repository. Pflegen Sie von Tag 1 an eine saubere `.gitignore`.

4. Erst Pullen, dann Pushen

Synchronisieren Sie vor Arbeitsbeginn und vor jedem Upload Ihren lokalen Stand per `git pull` mit dem Server, um unnötige Merge-Konflikte im Team frühzeitig zu vermeiden.

In vielen Unternehmen hat sich der Standard **Conventional Commits** etabliert, bei dem jede Commit-Nachricht mit einem klaren Präfix beginnt:

```
      # Struktur: <typ>(<optionaler Bereich>): <kurze Beschreibung> feat(rechner):
Quadratwurzel-Funktion mit Fehlerprüfung ergänzt fix(division): Absturz bei negativen
Gleitkommazahlen behoben docs(readme): Installationsanleitung für Python 3.12
aktualisiert refactor(core): Typannotationen (Type Hints) in allen Funktionen ergänzt
```

Fazit der Präsentation

Git hat die moderne Softwareentwicklung durch seine verteilte Architektur, seine Snapshot-Logik und das extrem schnelle Branching nachhaltig revolutioniert. Wer die Drei-Bäume-Architektur und das Zeiger-Modell von Branches verinnerlicht hat, beherrscht jedes Projekt – vom kleinen Python-Skript bis zur komplexen Unternehmensarchitektur.